

Topics in Error-Correcting Codes

Notes by Yael Kalai

MIT - 6.S953

Lecture 5: Interactive Coding (September 23, 2026)

Warning: This document is a rough draft, so it may contain bugs. Please feel free to email me with corrections.

Interactive Coding

Question: Can two parties interact over a noisy channel?

Example: Two players playing online chess over noisy channel.

We start by defining the notion of a binary protocol. In what follows we assume for simplicity that in each round both

Definition 1 (Binary protocol). In a binary protocol, Alice takes as input a sequence of functions $\Pi_A = \{\Pi_A^i\}_{i \in [\text{odd}]}$, where

$$\Pi_A^i : \{0, 1\}^{i-1} \rightarrow \{0, 1\}.$$

Similarly, Bob takes as input a sequence of functions $\Pi_B = \{\Pi_B^i\}_{i \in [\text{even}]}$, where

$$\Pi_B^i : \{0, 1\}^{i-1} \rightarrow \{0, 1\}.$$

The output of the binary protocol (Π_A, Π_B) is a sequence $(w_1, \dots, w_k)$ such that:

- For every odd $i \in [k]$, $\Pi_A^i(w_1, \dots, w_{i-1}) = w_i$.
- For every even $i \in [k]$, $\Pi_B^i(w_1, \dots, w_{i-1}) = w_i$.

Remark. Note that in the above we assume that each party sends a single bit per round. This is without loss of generality since we can send one bit at a time, at the cost of increasing the communication complexity by a factor of 2. This of course, does not preserve the round complexity, but for now we will not worry about preserving the round complexity.

Remark. Note that the above assumes that the protocol is deterministic.¹

From now on we assume that the number of rounds, denoted by k is fixed (padding with dummy bits if necessary). Thus, we can associate with each such protocol a binary depth k protocol tree.

Schulman [1] was the first to study the question of error correction in the interactive setting.

¹ This is without loss of generality since we can fix the parties' randomness.

Definition 2 (α -Error-Resilient Interactive Coding). An α -error-resilient interacting coding scheme converts any k -message protocol (Π_A, Π_B) into a $N = N(k)$ -message protocol (σ_A, σ_B) together with two functions (f_A, f_B) such that the following holds: Fix any vectors $(a_1, \dots, a_N)$ and $(b_1, \dots, b_N)$ for which:

- For every odd $i \in [N]$, $a_i = \sigma_A(a_1, \dots, a_{i-1})$.
- For every even $i \in [N]$, $b_i = \sigma_B(b_1, \dots, b_{i-1})$.
- $|\{i \in [N] : a_i \neq b_i\}| \leq \alpha N$.

Then $f_A(a_1, \dots, a_N) = f_B(b_1, \dots, b_N) = \text{Output}(\Pi_A, \Pi_B)$.

We note that while we assume that the underlying protocol (Π_A, Π_B) each message is a single bit, this is not the case in the error-resilient protocol (σ_A, σ_B) . In the latter each a_i and b_i is a string of bits of constant size.

Definition 3 (Interactive Coding). An interactive coding scheme is one that is ϵ -error resilient for a constant $\alpha \in (0, 1)$ and where the number of bits communicated in (σ_A, σ_B) is only a constant times larger than the number of bits communicated in (Π_A, Π_B) .

Interactive Coding Constructions

Attempt 1: Encode each message in the prtoocol using a (non-interactive) error correcting code.

Drawback: This only corrects errors locally. It is good for random errors by repeating each sent bit $\omega(\log k)$ times.

Our focus is on adversarial error. So we need a **global** error-correction mechanism.

Tree Codes

We now define the notion of a tree code, first defined by Schulman [2]. This is an online code which can be used for encoding binary protocols.

Definition 4. A tree code, T , is an online code with,

$$T : \{0, 1\}^* \rightarrow \Sigma^*,$$

such that the following conditions hold:

- $|x| = |y| \implies |T(x)| = |T(y)|$.
- For every $x \in \{0, 1\}^*$, $T(x)$ is the prefix of $T(x\|0)$ and $T(x\|1)$.
- There exists a constant $c \in \mathbb{N}$ such that $|\Sigma| = 2^c$.²

Note that by the definition of a tree code, for every $k \in \mathbb{N}$, every $x \in \{0, 1\}^k$, and every $i \in [k]$, $T(x_1, \dots, x_k)_{[1:i]}$ depends only on $(x_1, \dots, x_i)$.³

Definition 5. The rate of a tree code $T : \{0, 1\}^* \rightarrow \Sigma^*$, where $\Sigma = 2^c$, is $R = \frac{1}{c}$.

² Without loss of generality we think of

$\Sigma = \{0, 1\}^c$.

³ Here and below, a subscript $[j : t]$ on an encoded or received string denotes the concatenation of its output blocks $j, \dots, t$.

How should we define distance? Take any two conversations $x, y \in \{0, 1\}^k$. Suppose that $(x_1, \dots, x_{i-1}) = (y_1, \dots, y_{i-1})$ then

$$T(x)_{[1:(i-1)]} = T(y)_{[1:(i-1)]}$$

Thus the relative distance of a tree code is defined as follows:

Definition 6. A rate R tree code T has relative distance δ if

$$\forall x, y \in \{0, 1\}^k, \forall i \in [k], \quad x_i \neq y_i \implies \Delta(T(x), T(y)) \geq \frac{\delta(k-i+1)}{R}.$$

Theorem 7 (Tree codes imply interactive coding). *Suppose that T is a tree-code of rate $R = 1/c$ and relative distance $\delta > 0$, where c is a positive integer. Then there is a generic deterministic transformation taking any binary protocol communicating k bits into a protocol communicating $12k/R$ bits, such that both parties recover the original transcript even if an adversary corrupts a $\delta/96$ fraction of the transmitted bits.*

At a high level, each party maintains a tentative vertex in the protocol tree of the underlying binary protocol, and uses a tree code to communicate its moves, together with the bit prescribed by its input at its current vertex. By decoding the received messages, each party obtains a guess for the other party's current vertex. If it believes that their vertices coincide, it moves down the tree according to the bit supplied by the owner of that vertex. If it believes that it is ahead of the other party on the same path, or that their vertices lie on different branches, it backtracks. Otherwise, it believes that it is behind the other party on the same path, and stays. Each move, including backtracking or staying, is reported in the next round. Importantly, the tree-code history only grows: backtracking in the protocol tree is recorded as a new move, rather than erasing a previous move.

When both parties decode correctly, they either advance together along the correct path or reduce the disagreement between their tentative paths. Incorrect decoding can disrupt this progress, but the tree-code distance guarantee allows us to charge such disruptions to channel corruptions. Consequently, as long as the fraction of corrupted bits is sufficiently small, the parties make enough progress to recover the original transcript.

Proof. Set

$$N = 2k, \quad L = 3c = \frac{3}{R}, \quad \alpha = \frac{\delta}{3}.$$

N is the number of rounds in our interactive coding scheme, L is the length of each message sent. Our scheme uses the following tree code, over $\Sigma = \{0, 1\}^L$, with relative distance α .

The tree code. Let

$$\mathcal{A} = \{0, 1, \text{stay}, \text{back}\}, \quad \Sigma = \mathcal{A} \times \{0, 1\},$$

and fix a bijection $e : \Sigma \rightarrow \{0, 1\}^3$. Define the online code

$$\tilde{T}(h_1, \dots, h_t) = T(e(h_1) \parallel \dots \parallel e(h_t)).$$

This code emits L bits per input symbol, we thus view its output as a sequence of L -bit blocks.

We first verify that $\tilde{T}$ has relative bit distance at least α . Fix $h, h' \in \Sigma^t$ whose first disagreement is at position i . The binary strings

$$e(h_1) \parallel \cdots \parallel e(h_t) \quad \text{and} \quad e(h'_1) \parallel \cdots \parallel e(h'_t)$$

first disagree at some position $j \leq 3i$. Moreover, by the online property, $\tilde{T}(h)$ and $\tilde{T}(h')$ agree on their first $i - 1$ output blocks. Therefore,

$$\begin{aligned} \Delta(\tilde{T}(h)_{[i:t]}, \tilde{T}(h')_{[i:t]}) &= \Delta(\tilde{T}(h), \tilde{T}(h')) \\ &\geq \delta c(3t - j + 1) \\ &\geq \alpha L(3(t - i) + 1) \\ &\geq \alpha L(t - i + 1). \end{aligned} \tag{1}$$

The protocol tree. Let Π be the original protocol, and let $\mathcal{P}$ be its binary protocol tree. Each internal vertex is owned by one party, and the owner's input determines the outgoing bit.

We assume that every transcript has length k (padding with dummy bits if necessary). We further extend each leaf by a full binary tree so that the resulting tree has depth N . At every added internal vertex, Alice is the owner and her prescribed bit is 0. Recovering the first k bits of the extended execution suffices to recover the original transcript.

Identify every vertex with its root-to-vertex bit string. Write ε for the root, $|v|$ for the depth of v , and $\text{par}(v)$ for its parent, with

$$\text{par}(\varepsilon) = \varepsilon.$$

For $P \in \{A, B\}$, let

$$b_P(v) = \begin{cases} \text{the bit prescribed by } P\text{'s input at } v, & \text{if } P \text{ owns } v, \\ 0, & \text{otherwise.} \end{cases}$$

Define the move operation by

$$\text{Move}(v, a) = \begin{cases} v0, & a = 0, \\ v1, & a = 1, \\ v, & a = \text{stay}, \\ \text{par}(v), & a = \text{back}. \end{cases}$$

For a history

$$h = ((a_1, b_1), \dots, (a_t, b_t)) \in \Sigma^t,$$

$\text{vertex}(h)$ is a vertex on the protocol tree, defined by recursion, as follows:

$$u_0 = \varepsilon, \quad u_j = \text{Move}(u_{j-1}, a_j) \quad \text{for } j \in [t],$$

and set

$$\text{vertex}(h) = u_t, \quad \text{Bit}(h) = b_t.$$

The simulation. At each round t , we associate with each party $P \in \{A, B\}$ a vertex v_P^t , and we denote by a_P^t the move it makes at round t . Initialize

$$v_P^0 = \varepsilon, \quad a_P^0 = \text{stay}.$$

Each party also maintains an initially empty history over Σ .

The simulation consists of N rounds, where in each round both parties transmit a message simultaneously. In round t , party P , with the other party denoted by Q , performs the following steps:

1. Append

$$h_P^t = (a_P^{t-1}, b_P(v_P^{t-1}))$$

to its history,⁴ and send the t -th output block of

$$\tilde{T}(h_P^1, \dots, h_P^t).$$

This block consists of L bits.

2. Compute (a_P^t, v_P^t) from the (possibly corrupted) history blocks that it received from the other party, as follows:

- (a) Let $z_Q^t \in \{0, 1\}^{L \cdot t}$ be the concatenation of all blocks received from Q through round t . Compute

$$\hat{h}_Q^{(t)} \in \arg \min_{h \in \Sigma^t} \Delta(\tilde{T}(h), z_Q^t),$$

breaking ties according to a fixed rule. Set

$$\hat{v}_Q = \text{vertex}(\hat{h}_Q^{(t)}), \quad \hat{b}_Q = \text{Bit}(\hat{h}_Q^{(t)}).$$

- (b) Choose a_P^t as follows:

- If $v_P^{t-1} = \hat{v}_Q$, then set $a_P^t = b_P(v_P^{t-1})$ if P owns this vertex, and set $a_P^t = \hat{b}_Q$ otherwise.
- If v_P^{t-1} is a proper ancestor of $\hat{v}_Q$, set $a_P^t = \text{stay}$.
- Otherwise, set $a_P^t = \text{back}$.

Then update

$$v_P^t = \text{Move}(v_P^{t-1}, a_P^t).$$

Note that the histories are append-only: a move back changes the protocol-tree vertex but does not erase any history symbol.

After round N , each party outputs the first k bits of its vertex, or outputs failure if its vertex has depth less than k . Any padding is then removed.

Each round communicates $2L$ bits, so the total communication is

$$M = 2LN = 6cN = 12ck. \quad (2)$$

Thus, there is a $12c$ overhead in communication complexity; in other words, the rate of the interactive coding scheme is $\frac{1}{12c} = \frac{R}{12}$.

In the next lecture we prove the correctness of this protocol.

⁴ Recall that $a_P^{t-1} \in \{0, 1, \text{stay}, \text{back}\}$ is the move that party P made after decoding the message that it received from Q in round $t-1$, which caused P to move to vertex v_P^{t-1} .

References

- [1] Leonard J. Schulman. Communication on noisy channels: A coding theorem for computation. In *Proceedings of the 33rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 724–733. IEEE, 1992.
- [2] Leonard J. Schulman. Deterministic coding for interactive communication. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing (STOC)*, pages 747–756. ACM, 1993.